Analyse et Valorisation de Corpus Pédagogiques Textuels

Pôle Intelligence Artificielle

Projet No.2

Réalisé par :
Moncef AIT BELKACEM
Youssef BLAICH
Anas EL HABOUSSI
Zakaria EL OUAHIDI
Moez TRIKI

1er juin 2025

Table des matières

1 Introduction **1**
1.1 Présentation du client 1
1.2 Problème posé 1

2 État de l'Art **2**
2.1 Introduction 2
2.2 État de l'Art 2
2.2.0.1 Prétraitement 2
2.2.0.2 Génération du QCM 2
2.2.0.3 Métriques d'évaluation 3
2.2.0.4 Utilisation de Pipelines 3
2.2.0.5 Benchmark des modèles pour la génération de QCM 3
2.3 Conclusion 4

3 Description du Travail Réalisé **5**
3.1 Tâche 1 : Étude de l'état de l'art 5
3.2 Tâche 2 : Architecture de génération des QCM 5
3.3 Tâche 3 : Scraping (collecte de données) 5
3.4 Tâche 4 : Pipeline Text Chunker 6
3.5 Tâche 5 : Pipeline Concept Extractor 7
3.6 Tâche 6 : Pipeline Concept Combiner 7
3.7 Tâche 7 : Pipeline Question/Answer Generator 8
3.8 Tâche 8 : Pipeline Distractor Generator 9
3.9 Tâche 9 : Conception des métriques d'évaluation 9

4 Conclusion **10**
4.1 En résumé 10
4.2 Remerciements 10

5 Compléments **12**
5.1 Contribution – Moez 12
5.2 Contribution - Moncef 13
5.3 Contribution – Youssef 13
5.4 Contribution – Anas 14
5.5 Contribution – Zakaria 15

Annexes **16**

Chapitre 1

Introduction

1.1 Présentation du client

CleverLearn

CleverLearn est une startup innovante incubée au sein de **21st**, évoluant à l'intersection de l'éducation et de l'intelligence artificielle, avec pour ambition d'optimiser la progression des apprenants à tous les niveaux : scolaire, supérieur et professionel.

CleverLearn développe des modèles et des outils intelligents visant à améliorer la compréhension et l'apprentissage, en rendant les parcours pédagogiques plus efficaces, interactifs et personnalisés.

Sa vocation est de transformer les pratiques éducatives grâce à l'intelligence artificielle, afin de répondre aux besoins spécifiques de chaque apprenant, quel que soit son domaine de formation.

Mission

CleverLearn a pour mission de réinventer l'expérience d'apprentissage et de formation en concevant des solutions intelligentes qui s'intègrent naturellement aux environnements numériques actuels. En tirant parti des dernières avancées en intelligence artificielle, CleverLearn ambitionne d'offrir des parcours personnalisés, permettant à chaque utilisateur de progresser à son rythme et selon ses objectifs spécifiques.

1.2 Problème posé

Dans le cadre du développement de solutions pédagogiques innovantes, **CleverLearn** s'intéresse à l'exploitation de corpus pédagogiques textuels, tels que des cours ou des supports de formation.

L'objectif est d'extraire automatiquement, à partir de ces corpus, des référentiels de compétences, puis de générer des activités pédagogiques adaptées, comme des exercices ou des questionnaires à choix multiples (QCM), afin d'évaluer le niveau de compréhension des apprenants.

Le défi consiste à concevoir et à développer plusieurs **pipelines robustes**, capables d'assurer :

— l'extraction fiable des concepts et compétences clés,
— la génération automatique de QCM pertinentes,
— l'adaptabilité à divers domaines de connaissance,
— la qualité pédagogique des éléments générés.

Chapitre 2

État de l'Art

2.1 Introduction

L'intention de cette synthèse est de brosser un panorama des approches actuelles en génération automatique de questions à choix multiples (QCM). Elle n'a pas vocation à être exhaustive : le domaine évolue rapidement, porté entre autres par la montée en puissance des grands modèles de langage (LLM). Notre but est donc plutôt de dégager, d'organiser et d'éclairer les grands principes, outils et techniques que nous prévoyons d'employer en totalité ou en partie dans notre propre travail.

Toutes les étapes mentionnées ne seront pas forcément reprises telles quelles ; certaines serviront uniquement de cadre théorique, d'autres seront ajustées ou simplifiées pour répondre à nos contraintes et objectifs. De ce fait, cet état de l'art joue un double rôle : socle conceptuel pour guider la conception et point de repère pour motiver nos choix, tout en reconnaissant qu'il existe d'autres pistes non traitées ici.

2.2 État de l'Art

2.2.0.1 Prétraitement

La plupart des recherches commencent par extraire de vastes volumes de textes depuis Wikipédia ou d'autres encyclopédies ouvertes, constituant ainsi un corpus riche, neutre et libre d'accès. Le contenu brut est ensuite nettoyé : suppression des balises HTML, élimination des entêtes et passages redondants, uniformisation générale. Un passage crucial de ce prétraitement est la résolution de coréférence, qui remplace les pronoms par les groupes nominaux correspondants, rendant ainsi le texte plus clair et cohérent. Certains auteurs transforment aussi le résultat en Markdown ; cette mise en forme facilite la distinction entre titres et paragraphes et s'accorde mieux avec les modèles de type texte-vers-texte.

2.2.0.2 Génération du QCM

Le texte ainsi préparé est segmenté en unités sémantiques, les chunks, déterminées selon la similarité lexicale ou le thème traité. Dans l'approche de l'article **[1]** , chaque chunk reçoit un poids reflétant sa cohérence interne afin d'aiguiller l'extraction des idées centrales. (C'est d'ailleurs une super idée que nous avons décidé d'utiliser dans notre projet). À l'intérieur de ces segments, les candidats-réponses sont identifiés par diverses techniques : reconnaissance d'entités nommées (NER), extraction de groupes nominaux, sélection d' n-grams selon des règles syntaxiques ou encore filtrage statistique via TF-IDF ou des embeddings denses. On obtient ainsi un inventaire d'éléments chacun accompagné de son contexte précis.

Deux grandes stratégies régissent ensuite la génération des questions. La méthode answer-aware choisit d'abord la réponse cible, puis un modèle génératif comme T5, BERT ou BART, par exemple et élabore ainsi la question correspondante. À l'inverse, l'approche answer-agnostic produit question et réponse simultanément, sans contrainte initiale. Les modèles entraînés sur SQuAD restent solides, mais les LLM généralistes comme ChatGPT s'imposent pour la clarté, la cohérence et la qualité de structuration des QCM. Tanguy Raynaud dans son article **[2]** partir de bases de connaissances montre même que, selon la thématique, on peut varier de modèle ; sur les sujets littéraires, ChatGPT l'emporte nettement grâce à sa finesse rédactionnelle.

La fabrication des distracteurs vient juste après. Deux méthodes dominent : soit puiser, dans le même chunk, des expressions proches de la bonne réponse qu'on reformule pour qu'elles restent crédibles tout en étant incorrectes, soit recourir à des ressources externes WordNet, DBpedia, Word2Vec ou un LLM pour générer des alternatives fausses mais plausibles. Les grands modèles servent également à reformuler ou filtrer ces distracteurs afin d'en assurer la cohérence.

2.2.0.3 Métriques d'évaluation

L'évaluation combine des critères automatiques et humains. Côté automatique, BLEU et ROUGE mesurent la qualité textuelle, Exact Match vérifie la correspondance exacte avec une référence, et F1 apprécie le chevauchement de tokens. Certaines études ajoutent la similarité cosinus : entre bonne réponse et distracteurs pour s'assurer qu'ils ne se recoupent pas, et entre question et texte source pour juger la pertinence. Côté humain, les items sont jugés sur la grammaire, la pertinence de l'énoncé, la validité de la bonne réponse, la fausseté et la vraisemblance des distracteurs, la difficulté et la qualité globale, le tout sur échelles notées qui permettent des statistiques robustes. Dans ces comparaisons, ChatGPT génère régulièrement les QCM les plus lisibles et pertinents.

2.2.0.4 Utilisation de Pipelines

Des articles, comme celui de Renato Miguel Francisco de Matos **[3]**, regroupent ces étapes : nettoyage et coréférence, première pipeline, extraction des réponses via NER ou TF-IDF, deuxième pipeline, génération des questions par Transformer fine-tuné, troisième pipeline, création des distracteurs avec BART ou des ressources sémantiques externes quatrième et dernière pipeline.

2.2.0.5 Benchmark des modèles pour la génération de QCM

Les études récentes distinguent deux grandes familles de solutions : la première est l'utilisaiton des transformeurs open-source spécialisés, fine-tunés sur des corpus d'apprentissage, et la duexième les LLM généralistes exploités en mode zero- ou few-shot.

Transformeurs spécialisés (T5, BART, BERT2BERT, Flan-T5). Fine-tunés sur SQuAD ou LEAF, ces modèles produisent des QCM corrects à faible coût d'inférence ; T5-base atteint en général des bons résultats sur les ensembles de référence. Les variantes retrieval-augmented (BART + RAG ou DG-KG) améliorent surtout la génération de distracteurs grâce à l'injection de connaissances externes : **[4]**. –> Limites : contenu parfois télégraphique, qualité linguistique en deçà des LLM.

LLM généralistes (ChatGPT / GPT-3.5, GPT-4o, Llama, Mixtral). Sans fine-tuning, ChatGPT obtient des scores automatiques comparables aux meilleurs transformeurs mais surpasse nettement en évaluation humaine : lisibilité notée 4,3/5 vs 3,7 pour T5, validité des distracteurs 4,1/5 **[5]**. Différents articles établissent de meilleurs résultats pour la génération de QCM avec l'utilisation des modèles d'Open AI :**[6] [7] [8] [9]**

Les modèles open-source récents (Llama-2-Chat-70B, Mixtral-8x7B-Instruct) restent en retrait : qualité textuelle correcte mais distracteurs moins crédibles, et tendance accrue aux hallucinations thématiques.

2.3 Conclusion

Pour résumer, la génération automatique de QCM s'appuie désormais sur un éventail de techniques allant de la linguistique traditionnelle aux modèles de langage les plus récents. De la constitution du corpus jusqu'au contrôle qualité, chaque maillon du processus dispose d'outils éprouvés, souvent couplés au sein d'architectures modulaires. Résolution de coréférence, segmentation sémantique, extraction ciblée de concepts, Transformers fine-tunés : autant de méthodes diverses que nous exploiterons largement dans notre propre démarche.

Chapitre 3

Description du Travail Réalisé

3.1 Tâche 1 : Étude de l'état de l'art

Déroulement Après réception d'une bibliographie succincte fournie par le client, nous nous sommes attelés à la lecture assidue de ces supports. Nous avons de plus consulté les références de nos encadrants. Cette étude préliminaire nous a permis de nous acclimater avec la problématique relevée par le client et de nous renseigner quant aux possibles pistes à explorer.

Livrables des rapports succincts des éléments pertinents de chaque article ont été effectués. Nous avons en particulier retenu les méthodologies énoncés dans la thèse de Renato Miguel Francisco de Matos **[3]**

3.2 Tâche 2 : Architecture de génération des QCM

Déroulement

Mise en place d'une architecture structurée en cinq pipelines successifs : trois dédiés au prétraitement du texte, suivis de deux pipelines consacrés à la génération automatique des questions, des réponses correctes et des distracteurs. La partie état de l'art nous a fortement inspiré. Youssef, Moez et Moncef se sont accordés en 2h sur l'enchaînement des pipelines. Heureusement que les résultats étaient immédiatement concluants post-implémentation de cette structure là (qui persiste dans le projet final). Moncef était responsable de l'architecture commune des pipelines, en particulier la classe mère `Pipeline` et la classe `QuizGenerator` chargée de gérer le cheminement des données à travers les différents pipelines.

Livrables Les classes Pipeline et QuizGenerator dans le repositoire final.

Éléments techniques L'algorithme retenu est le suivant, nous avons choisi d'avoir cinq pipelines dans la liste.

```
input pipelines : List[Pipeline]
input corpus : Corpus
tmp ← corpus
for pipeline in pipelines do
    tmp ← pipeline.process(tmp)
end for
return tmp
```

3.3 Tâche 3 : Scraping (collecte de données)

Déroulement

Cette tâche avait pour but de collecter automatiquement du contenu pédagogique à partir de pages Wikipédia, dans le but de constituer un corpus utilisable par la suite pour l'extraction de concepts et la génération de QCM. Pour cela, nous avons développé un scraper personnalisé capable de récupérer une page HTML, d'en extraire le contenu utile, puis de le convertir en format Markdown lisible.

La collecte s'est déroulée en plusieurs étapes : récupération de la page via une requête HTTP, suppression des éléments inutiles (menus, images, scripts, références, etc.), puis transformation du corps de la page en texte brut structuré. Le scraping a été appliqué à plusieurs pages thématiques, choisies pour leur richesse en contenu éducatif.

Éléments techniques :

- **Bibliothèques utilisées** :
  - `requests` : pour effectuer les requêtes HTTP et récupérer le contenu HTML brut des pages.
  - `BeautifulSoup (bs4)` : pour parser le HTML et sélectionner précisément les éléments pertinents.
  - `markdownify` : pour convertir le HTML nettoyé en Markdown lisible et plus facile à manipuler.
- **Nettoyage du HTML** :
  - Suppression de l'en-tête, du pied de page, des menus de navigation, des scripts, styles, images, boutons, formulaires, tableaux et éléments de métadonnées.
  - Suppression des sections non pertinentes comme *References*, *External Links*, *Bibliography*, *See Also*, etc.
- **Gestion des erreurs** :
  - Vérification du code de retour HTTP (status code) pour ignorer les pages inaccessibles.
  - Gestion des exceptions lors de l'appel à l'API de Wikipedia pour éviter les crashs en cas de perte de connexion ou de contenu non conforme.
- **Format des données récupérées** :
  - Le contenu final est exporté en Markdown, ce qui permet une visualisation simple et une compatibilité directe avec les autres modules de traitement textuel du projet.

3.4 Tâche 4 : Pipeline Text Chunker

Déroulement :

Cette tâche avait pour objectif de découper automatiquement un texte long (ex. : cours, article Wikipédia) en segments cohérents appelés *chunks*, pour les rendre exploitables par la suite dans le pipeline (extraction de concepts, génération de QCM).

Nous avons d'abord conçu une version naïve du *Text Chunker*, reposant sur une stratégie simple de découpage à chaque retour à la ligne `\n`, à condition qu'un seuil minimal de caractères soit atteint entre deux coupures (`minimum_chunk_length`). Cette méthode assurait une première séparation rapide mais ne garantissait pas la cohérence sémantique entre les morceaux.

Afin d'améliorer significativement la qualité des segments extraits, nous avons ensuite conçu une version *sémantique* du *Text Chunker*. Celle-ci utilise des modèles de langage pour regrouper les phrases en paragraphes cohérents sur le plan thématique. Le découpage ne repose plus uniquement sur des critères structurels (comme la taille ou les sauts de ligne), mais sur la similarité sémantique entre phrases successives, permettant ainsi de former des unités de sens complètes et homogènes. Cela est particulièrement crucial pour les modèles de génération de QCM, qui nécessitent un contexte logique pour produire des questions pertinentes.

Éléments techniques :

- **Méthode naïve** :

  - Découpage basé sur les retours à la ligne `\n` en respectant une taille minimale.
  - Implémentation dans la classe `TextChunker`, intégrée à l'architecture des pipelines via héritage.
  - Vérification de la cohérence en recalculant la longueur totale du texte à partir des morceaux.
- **Méthode sémantique** :
  - Utilisation de modèles de plongement (ex. `SentenceTransformer`, `all-MiniLM-L6-v2`) pour encoder les phrases.
  - Calcul de similarités cosinus entre vecteurs de phrases.
  - Fusion des phrases en paragraphes dès lors que leur similarité reste au-dessus d'un certain seuil, garantissant la continuité thématique.
  - Possibilité de configurer dynamiquement la longueur cible et le degré de cohérence exigé.

3.5 Tâche 5 : Pipeline Concept Extractor

Déroulement :

Cette tâche consiste à extraire automatiquement les concepts clés de chaque paragraphe d'un document, dans le but de structurer l'information et d'alimenter les étapes suivantes de génération de questions. Pour cela, nous avons mis en place un pipeline `ConceptExtractor` qui interagit avec un modèle de langage via une API, afin de produire, pour chaque bloc de texte, une liste limitée de concepts pertinents.

Le processus repose sur un traitement parallèle par threads pour améliorer la rapidité d'exécution. Chaque paragraphe est soumis individuellement au modèle, avec une consigne de génération imposant un nombre maximal de concepts. Les résultats sont ensuite validés automatiquement et stockés pour une réutilisation ultérieure.

Éléments techniques :

- **Modèle utilisé** : accès à l'API d'OpenAI via la classe `OpenAI_client`, avec appels structurés par paires de messages (développeur + utilisateur).
- **Parallélisation** :
  - Utilisation de `ThreadPoolExecutor` du module `concurrent.futures` avec `NTHREADS = 20` pour traiter les paragraphes en parallèle.
  - Abandon de l'approche asynchrone (`asyncio`) au profit d'une solution multi-threads plus fiable dans ce contexte.
- **Format d'entrée** : liste de paragraphes extraits par le `TextChunker`.
- **Limitation du nombre de concepts** :
  - Le nombre maximal de concepts extraits par paragraphe est défini par `concept_limit`.
  - Cette contrainte est intégrée dans le prompt passé au modèle.
- **Validation** :
  - Vérification que le nombre de concepts par chunk respecte la limite imposée.
  - Journalisation d'un avertissement en cas de dépassement.
- **Sortie** : Liste de listes de chaînes de caractères, chaque sous-liste contenant les concepts extraits d'un paragraphe donné.

3.6 Tâche 6 : Pipeline Concept Combiner

Déroulement : L'objectif est de réduire la redondance conceptuelle générée par le module d'extraction. À partir d'une liste de listes de chaînes représentant divers concepts, il faut produire un ensemble condensé, trié par importance. Le modèle `paraphrase-multilingual-MiniLM-L12-v2`

vectorise chaque concept en tenant compte du contexte phrastique. Les vecteurs obtenus sont comparés à l'aide de la similarité cosinus, ce qui donne une matrice de similarité S. Cette matrice est convertie en matrice de distance via $D = 1 - S$. La partie triangulaire supérieure de D sert de matrice d'adjacence ; l'algorithme de regroupement hiérarchique agglomératif est alors appliqué grâce à la fonction `linkage` de `scipy.cluster.hierarchy`. Les clusters ainsi formés il faut extraire un représentant par cluster **Éléments techniques :**

- Utilisation de modèles de plongement (ex. `SentenceTransformer`, `all-MiniLM-L6-v2`) pour encoder les phrases.
- Calcul de similarités cosinus entre vecteurs de phrases.
- Algorithme linkage de la bibliothèque scipy.cluster.hierarchy
- **Format d'entrée** : liste de listes de concepts extraits par le `Concept Extractor`
- **Sortie** : Liste de concept.

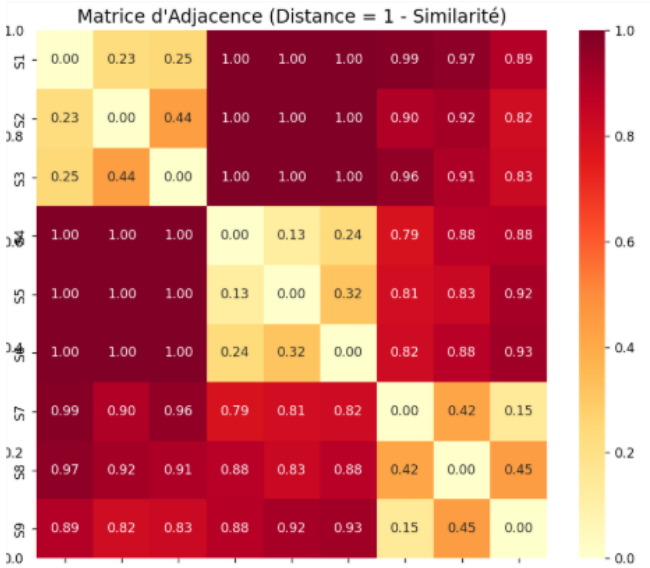

FIGURE 3.1 – Visualisation des clusters de concepts

3.7 Tâche 7 : Pipeline Question/Answer Generator

Déroulement :
L'objectif de cette tâche est de générer automatiquement des paires question/réponse à partir de concepts extraits dans les étapes précédentes du pipeline. Chaque concept (ou groupe de concepts) est transmis à un modèle de langage qui formule une question pertinente et fournit une réponse correspondante. L'ensemble de ces questions/réponses constitue la base à partir de laquelle seront ensuite créés des QCM.
Le pipeline `QuestionAnswerGenerator` a été conçu pour prendre en entrée une liste de concepts, les transformer en messages structurés, puis interroger une API d'un LLM . Les résultats sont stockés sous forme d'objets structurés (instances du modèle `QuestionAnswer`) afin d'assurer la cohérence et la compatibilité avec les étapes suivantes.
Éléments techniques :

- **Modèle utilisé** : appel à l'API d'OpenAI via une classe `OpenAI_client`, encapsulant les requêtes au modèle.
- **Structure des messages** :
  - Chaque appel au modèle est constitué d'un message développeur (instructions de génération) et d'un message utilisateur (contenant le concept cible).

— **Format d'entrée** : liste de concepts thématiques (chaînes de caractères), groupés par paragraphes (liste de listes).
— **Format de sortie** : liste de tuples `(question, réponse)` ; chaque élément étant validé par un modèle Pydantic (`QuestionAnswer`) garantissant la conformité du format.
— **Intégration** : ce pipeline est exécuté après le `ConceptCombiner`, et ses résultats sont sauvegardés dans l'infrastructure d'indexation pour réutilisation dans la génération des QCM.

3.8 Tâche 8 : Pipeline Distractor Generator

Déroulement :
Cette tâche vise à générer automatiquement les distracteurs, c'est-à-dire les mauvaises réponses plausibles à proposer dans un QCM. Elle intervient après la génération des paires question/réponse. Le pipeline prend en entrée une liste de questions avec leurs réponses correctes, et produit pour chacune un ensemble de distracteurs, de manière à former une question à choix multiple (QCM) complète.
Le processus repose sur l'interrogation d'un LLM, à qui l'on fournit des instructions spécifiques pour générer un certain nombre de distracteurs réalistes et non triviaux. Les résultats sont ensuite structurés sous forme d'objets `MultipleChoiceQuestion`, eux-mêmes regroupés dans un objet `Quiz`.
Éléments techniques :
— **Modèle utilisé** : appel à l'API d'OpenAI via la classe `OpenAI_client`, avec gestion parallèle des requêtes pour optimiser le temps de génération.
— **Structure des messages** :
  — Chaque requête envoyée au modèle inclut un message développeur contenant les instructions pour générer au moins un certain nombre de distracteurs.
  — Le message utilisateur fournit la question et sa réponse correcte sous une forme structurée.
— **Format d'entrée** : liste de tuples `(question, réponse)`.
— **Format de sortie** :
  — Liste d'objets `MultipleChoiceQuestion`, contenant la question, la réponse correcte et une liste de distracteurs générés.
  — Encapsulation dans un objet `Quiz` pour assurer la compatibilité avec les autres modules d'évaluation.
— **Validation** : la structure des distracteurs est contrôlée via un modèle Pydantic (`Distractors`) qui garantit le respect du format attendu.

3.9 Tâche 9 : Conception des métriques d'évaluation

Pour évaluer la qualité des QCM générés automatiquement, nous avons défini trois axes principaux d'évaluation. Le premier concerne la pertinence des questions par rapport au contenu de l'article, afin de s'assurer qu'elles reposent sur des informations réellement présentes dans le texte. Le second axe porte sur la couverture de l'article, c'est-à-dire la capacité du quiz à balayer de manière équilibrée les différentes parties du document source. Enfin, le troisième axe vise à évaluer la qualité des distracteurs, c'est-à-dire les mauvaises réponses proposées, qui doivent être à la fois plausibles et discriminantes. Ces trois dimensions complémentaires permettent d'apprécier globalement la cohérence, la diversité et la fiabilité des QCM générés.

Chapitre 4

Conclusion

4.1 En résumé

Ce projet nous a permis d'explorer en profondeur les enjeux liés à la génération automatique de QCM à partir de corpus pédagogiques textuels, dans un contexte où l'intelligence artificielle joue un rôle de plus en plus central dans la transformation des pratiques éducatives. En nous appuyant sur un état de l'art riche et en constante évolution, nous avons conçu une architecture modulaire articulée autour de pipelines spécialisés, chacun remplissant une fonction précise dans le traitement du texte, l'extraction de concepts et la génération d'éléments évaluatifs.
La mise en œuvre des différentes tâches nous a permis de combiner rigueur technique et créativité algorithmique. Qu'il s'agisse du scraping de contenus, du découpage sémantique, de l'extraction et de l'agrégation de concepts, ou encore de la génération de questions et distracteurs à l'aide de modèles de langage, chaque étape a été pensée pour maximiser la cohérence pédagogique et la qualité des QCM produits. L'intégration de métriques automatiques, comme la similarité sémantique ou les scores de modèles de question-réponse, a par ailleurs permis une évaluation fine des résultats.
Au-delà des résultats obtenus, ce projet nous a permis de développer des compétences clés en ingénierie d'IA, en traitement automatique du langage naturel, et en conception de systèmes robustes à base de LLM. Il constitue une base solide pour de futurs travaux, notamment en vue d'automatiser la génération de contenus éducatifs ou de les adapter à différents niveaux et domaines de formation. Des perspectives d'amélioration sont également envisageables, notamment en matière d'optimisation des prompts, de diversité des formats de questions, ou d'adaptation aux préférences individuelles des apprenants.

4.2 Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude à Ibrahim, fondateur de la start-up CleaverLearn, pour le temps précieux consacré à notre équipe, la qualité de ses retours toujours constructifs et la manière dont il a su challenger chacune de nos idées afin d'enrichir, sans cesse, les pistes explorées.
Nos remerciements vont ensuite à nos encadrants du pôle projet, Wassila Ouerdane et Jean-Philippe Poli, dont l'accompagnement constant, les conseils avisés et le suivi régulier ont permis de mener ce travail dans la bonne direction, en identifiant les points d'amélioration nécessaires à chaque étape.
Enfin, nous adressons un sincère merci à nos encadrantes APP API, Camille Gandiolle et Céline Fondeur, dont les observations, qu'il s'agisse de la forme de nos présentations ou de l'organisation globale du projet, ont grandement contribué à hisser notre travail

vers un niveau d'exigence supérieur.
À toutes et tous, merci pour votre confiance, votre disponibilité et votre engagement à nos côtés.

Chapitre 5

Compléments

5.1 Contribution – Moez

Dans le cadre de ce projet, j'ai eu l'occasion de contribuer activement à plusieurs tâches techniques, en particulier sur le traitement du texte et l'évaluation de la qualité des QCM générés. Concernant le **pipeline Text Chunker** (70 %), mes collègues ont d'abord mis en place une version initiale reposant sur un découpage naïf du texte, basé sur les retours à la ligne et une contrainte de longueur minimale. J'ai ensuite repris ce travail pour l'améliorer progressivement en développant une version *sémantique* du chunker. En m'appuyant sur le modèle `all-MiniLM-L6-v2` (Sentence-BERT), j'ai mis en place un système de regroupement des phrases fondé sur leur similarité sémantique, mesurée par la distance cosinus entre leurs embeddings. Cela a permis de former des paragraphes plus homogènes sur le plan du contenu, tout en respectant les contraintes de longueur imposées par les modèles en aval. Cette tâche m'a pris environ une séance complète de travail, durant laquelle j'ai aussi dû ajuster dynamiquement les seuils de similarité et tester plusieurs configurations pour éviter à la fois les redondances et les coupures trop brutales dans le texte.
Sur le plan de l'évaluation, j'ai eu la responsabilité de concevoir et implémenter la **métrique de pertinence des questions** générées. Pour cela, j'ai utilisé le modèle `roberta-base-squad2`, spécialisé dans la tâche de question answering. L'idée était d'évaluer, pour chaque question, la capacité du modèle à retrouver une réponse plausible dans un des paragraphes du texte source. J'ai ainsi développé un système qui parcourt l'ensemble des chunks pour identifier celui qui maximise le score de confiance du modèle. Ce score est interprété comme un indicateur de pertinence : plus il est élevé, plus la question est cohérente avec le texte. J'ai aussi utilisé ces données pour analyser la **couverture du document** en visualisant la distribution des chunks sollicités. Cette partie m'a demandé environ deux séances de travail, réparties entre le développement de la métrique, les tests sur différents corpus et la production de visualisations interprétables (histogrammes, diagrammes circulaires).
Enfin, j'ai participé à la **conception globale des métriques d'évaluation** (33 %), à la relecture des résultats produits, ainsi qu'à la rédaction des sections du rapport correspondant aux modules dont j'ai eu la charge. D'autres tâches annexes ont été réparties au fil du projet, et j'ai également contribué à la **recherche bibliographique sur l'état de l'art**, en collaboration avec l'ensemble du groupe, durant deux séances dédiées. Ce travail m'a permis de mieux comprendre les approches actuelles en génération de QCM à base de modèles de langage, tout en affinant ma compréhension de l'articulation entre extraction d'information, génération contrôlée et évaluation automatique.

5.2 Contribution - Moncef

Étant à l'aise en développement python et en gestion de repositoire git, j'ai pris l'initiative en début de projet de standardiser au maximum nos interactions avec l'outil git. Ainsi, chaque membre du groupe a dû créer sa branche et je m'assurais de l'intégration régulière et progressive des fonctionnalités implémentées. Par la suite, je me suis attelé, comme tous mes camarades, à la lecture des différents articles mis à notre disposition. Dès que l'architecture générale se faisait de moins en moins elusive, j'ai consacré quelques heures à la classe `QuizGenerator` et `Pipeline`.
Le deuxième week end après le début du projet, je me suis chargé de la mise en place des classes de scraping wikipedia, ainsi que de la classe `OpenAI_client`. A cet effet, nous nous sommes retrouvés moi et Youssef afin de découvrir l'api `OpenAI` et d'en tirer partie. Nous nous sommes aussi accordés sur les librairies `requests` et `beautifulsoup` pour la partie scraping. Après lecture de quelques tutoriels et quelques tentatives, nous avions fini. Nous avons aussi implémenté une version naïve du text chunker.
Anas, Youssef et moi même avons pendant une séance implémenté le concept-extractor en utilisant la classe `OpenAI_client`. Ayant découvert des temps d'attente inacceptables pour cette première implémentation, j'ai dégagé trois heures afin d'étudier la programmation concurrente en python et pallier à ce problème. J'ai considéré asyncio et le multiprocessing, mais finalement le multithreading était plus simple à implémenter et assurait une accélération de x8 (nombre de threads choisi) qui suffisait.
Lors de discussions fort constructives avec zakaria et moez, nous nous sommes accordés sur le fonctionnement du concept combiner. Zakaria s'est ensuite empressé de l'implémenter. Le question-answer generator a été fait en une séance par zakaria et moi même.
De manière générale je me suis chargé de nombreux soucis techniques : environnement virtuel, nettoyage de code, infrastructure de logs, infrastructure de stockage des prompts et l'utilisation du module `argparse`.
En fin de projet, je me suis assuré de la qualité de la soutenance finale, en chronométrant diligemment les six soutenances test que nous avons effectuées.
Ce projet m'a permis de m'essayer à la programmation en équipe, le pair programming et la gestion de projet numérique. J'ai fort apprécié les expériences de nombreux outils que je retrouverais sans aucun doute dans la suite de mon parcours académique et professionnel.

5.3 Contribution – Youssef

Dans le cadre de ce projet, j'ai activement participé à des tâches variées allant de la phase de conception à la mise en œuvre technique, jusqu'à l'évaluation finale des résultats. J'ai commencé par consulter plusieurs articles scientifiques sur le sujet, ce qui m'a permis de bien comprendre les enjeux liés à la génération automatique de QCM et de m'inspirer pour les étapes suivantes du travail.
J'ai ensuite contribué de manière significative (à hauteur de 50 %) à la conception de l'architecture du système de génération. Après avoir échangé avec notre client, je me suis concerté avec mon collègue Moncef pour imaginer une structure modulaire et efficace. Nous avons progressivement élaboré une architecture en cinq étapes visant à limiter les appels API, réduire les hallucinations du modèle, améliorer l'exhaustivité des questions et assurer un découpage clair permettant de diagnostiquer facilement les éventuels dysfonctionnements. L'architecture retenue comprenait cinq pipelines enchaînés : découpage du texte (*text chunker*), extraction de concepts (*concept extractor*), combinaison de concepts (*concept combiner*), génération de questions-réponses et génération de distrac-

teurs. J'ai également participé à l'implémentation de cette architecture en développant les classes associées dans le code.
Par ailleurs, j'ai contribué à hauteur de 50 % à la partie technique liée au *scraping*, en collaboration avec Moncef, afin de collecter les données nécessaires à l'alimentation du système.
J'ai aussi pris part à l'amélioration du pipeline d'extraction de concepts (environ 33 %). J'ai optimisé une partie du code existant en réduisant sa complexité, et j'ai retravaillé les *prompts* envoyés au modèle à partir de l'analyse des résultats, en renforçant les consignes pour améliorer la qualité des concepts extraits.
Enfin, j'ai participé à la phase d'évaluation (33 %), en commençant par la conception des métriques adaptées pour juger la qualité des QCM générés. Je me suis appuyé sur plusieurs articles pour identifier des critères pertinents comme la difficulté, la pertinence ou encore l'exhaustivité, et j'ai contribué aux discussions sur les méthodes de mesure possibles. J'ai ensuite implémenté une première approche basée sur la probabilité du prochain *token* généré par le modèle. Après plusieurs tests, j'ai constaté que le LLM parvenait à répondre aux questions générées de manière très précise sans avoir besoin de consulter les documents sources, remettant ainsi en cause l'hypothèse fondatrice de cette méthode. J'ai présenté mes observations à mes collègues, ce qui a conduit à l'abandon de cette approche.

5.4 Contribution – Anas

J'ai participé à plusieurs étapes clés du projet, en intervenant tant sur le traitement du texte que sur la génération et l'évaluation des QCM.
En début de projet, j'ai contribué au **pipeline Text Chunker**, en travaillant sur la première version du découpage du texte en unités exploitables. Cette étape consistait à segmenter les documents en phrases ou en blocs sémantiquement cohérents à l'aide de bibliothèques telles que `nltk`, `spaCy` ou encore les modèles de type `transformers`. Cette structuration initiale a servi de base à l'ensemble des traitements suivants.
J'ai également été impliqué dans le **pipeline Concept Extractor**, où j'ai collaboré à la mise en place de méthodes d'extraction de concepts clés. J'ai exploré plusieurs approches telles que `TF-IDF`, `RAKE` ou `KeyBERT`, dans le but d'identifier les éléments significatifs du texte à transformer en questions.
Mon rôle a ensuite été central dans le **pipeline Question/Answer Generator**, où j'ai utilisé des modèles de type `T5` pour générer automatiquement des questions à choix multiples. J'ai notamment veillé à la cohérence grammaticale des questions, à la pertinence des réponses proposées, ainsi qu'à l'équilibre global des QCM produits.
En fin de projet, j'ai contribué à la **conception des métriques d'évaluation**, en définissant des critères objectifs pour apprécier la qualité des QCM. J'ai plus particulièrement pris en charge la **métrique fondée sur la similarité sémantique**, que j'ai conçue et implémentée à l'aide de `Sentence-BERT`. L'idée était de comparer, via la distance cosinus, la proximité sémantique entre la question et chacune des réponses proposées, dans le but de vérifier si la bonne réponse se distingue nettement des distracteurs.
Ces différentes contributions m'ont permis de mobiliser des compétences variées en NLP, en modélisation de texte et en évaluation automatique, tout en participant activement à chaque phase du projet.

5.5 Contribution – Zakaria

Dans ce projet, j'ai participé à plusieurs étapes du pipeline, notamment sur les parties liées à la structuration du texte et à l'enrichissement du contenu des QCM.
J'ai tout d'abord mis en place, dans notre premier *MVP* (produit minimum viable), une première version du **Text Chunker** fondée sur un découpage *naïf* des textes. Cette version reposait sur des règles simples, comme les retours à la ligne ou une longueur minimale, pour segmenter le texte en unités exploitables. Elle a permis de rapidement tester la faisabilité du pipeline en conditions réelles, avant les améliorations sémantiques apportées par la suite.
Mon apport principal a concerné le **Concept Combiner**. Cette étape vise à regrouper les concepts similaires extraits du texte, afin de réduire la redondance et d'obtenir des éléments plus pertinents pour la génération des questions. J'ai conçu une méthode de regroupement par similarité en utilisant le modèle `all-MiniLM-L6-v2` (Sentence-BERT), j'ai ensuite trouver et comparer différents algorithme de clustering afin de savoir lequel s'approcher le plus de notre cas et j'ai ainsi terminer cette pipeline en élaborant un clustering sémantique des concepts. Cela a amélioré la diversité des concepts tout en évitant les répétitions dans les QCM.
J'ai également contribué de manière significative au **Distractor Generator**. Ce module est chargé de produire des *distracteurs*, c'est-à-dire des réponses incorrectes mais plausibles pour accompagner la bonne réponse. J'ai exploré plusieurs stratégies pour cela : extraction d'éléments voisins dans le texte, génération automatique via un modèle de langage, puis sélection et filtrage en fonction de leur cohérence grammaticale et sémantique. L'objectif était d'introduire un *niveau de difficulté équilibré*, en évitant à la fois les distracteurs trop faciles ou trop ambigus.
De plus, j'ai contribué aux échanges collectifs autour de l'état de l'art, en apportant des exemples concrets et en participant activement aux décisions sur les choix techniques du pipeline. Enfin, étant à l'aise à l'oral, j'ai essayé de mener à bien les séances d'entrainements en proposant divers amélioration, que ça soit sur la façon de parler ou même sur notre support de présentation.

Répartition des tâches

Tâches	Personnes en charge
Étude de l'état de l'art	Tout le groupe
Architecture de génération des QCM	Moncef 50% / Youssef 50%
Scraping	Moncef 50% / Youssef 50%
Pipeline Text Chunker	Moez 70% / Zakaria 20% / Anas 10%
Pipeline Concept Extractor	Moncef 33% / Youssef 33% / Anas 33%
Pipeline Concept Combiner	Zakaria 80% / Moez 10% / Moncef 10%
Pipeline Question/Answer Generator	Anas 50% / Moez 50%
Pipeline Distractor Generator	Zakaria 60% / Moncef 40%
Conception métrique d'évaluation	Anas 33% / Youssef 33% / Moez 33%
Métrique utilisant roberta-base-squad2	Moez
Métrique utilisant la similarité sémantique	Anas
Métrique utilisant la probabilité du prochain token	Youssef

FIGURE 5.1 – Répartition indicative des contributions par membre